Systemizing Your AI: From Ad-Hoc Prompts to Repeatable AI Workflows

A Practitioner's Guide to CustomGPTs, Gems, and Claude Code Skills

---

Version 1.0 — March 2026

Table of Contents

1. Why Systemize at All?
2. The Anatomy of a System Prompt
3. Understanding the Three Platforms
4. Part I: OpenAI CustomGPTs — Building Persistent Chat Agents
5. Part II: Google Gemini Gems — Lightweight Specialists
6. Part III: Claude Code Skills — Prompt-as-Code for Developers
7. The Universal Framework: A Five-Layer Model
8. Writing Effective System Instructions
9. Knowledge Management Across Platforms
10. Testing and Iterating
11. Cross-Platform Comparison Matrix
12. Real-World Workflow: Building a "Workshop Launch Assistant"
13. Common Mistakes and How to Avoid Them
14. Next Steps

---

1. Why Systemize at All?

If you already write good prompts, you have solved the hardest part of working with AI. But there is a difference between writing a good prompt and building a good system. Consider this analogy: a talented chef can improvise a great meal, but a restaurant needs recipes, prep lists, and station assignments. Systemizing is how you turn improvisation into a reliable, repeatable operation.

Every time you open a fresh AI chat and paste the same paragraph of context — "I'm building a SaaS product, my audience is mid-market IT buyers, I want concise

answers with code examples" — you are paying a tax. It costs you time, it costs you tokens, and it introduces variance. On Monday you remember to mention your coding style preferences; on Thursday you forget, and the output drifts.

Systemizing solves three problems at once:

Consistency. A well-built CustomGPT, Gem, or Skill produces outputs that follow the same rules every time. You define the guardrails once, and they apply to every conversation.

Delegation. Once a workflow is systemized, you can hand it to a colleague, a team member, or even a scheduled automation. They do not need to know your prompt-engineering tricks; the system encodes them.

Composability. Individual systemized components can be chained together. A "research" Gem feeds into a "draft" CustomGPT, which feeds into a "review" Skill. Each link in the chain does one thing well.

The goal of this guide is not to teach you prompting — you already know that. The goal is to teach you the architecture: where instructions live, how they are loaded, what each platform calls them, and how to design them so they survive real-world use.

---

2. The Anatomy of a System Prompt

Before we look at platform-specific tools, we need a shared vocabulary. Regardless of whether you are building a CustomGPT, a Gem, or a Skill, you are always constructing some version of the same thing: a **system prompt** — a set of instructions that the AI reads before it reads your message.

A system prompt typically has four layers:

Identity. Who is the AI in this context? "You are a senior marketing strategist with 15 years of B2B SaaS experience." This sets the baseline behavior: vocabulary, reasoning depth, confidence level.

Task. What is the AI supposed to do? "Analyze the uploaded competitive report and produce a SWOT matrix." The more specific the task definition, the less the user needs to type each time.

Constraints. What should the AI avoid? "Never recommend strategies that require a budget above €5,000. Do not use jargon without defining it first." Constraints are boundaries. They prevent drift and keep outputs inside acceptable ranges.

Format. How should the output look? "Respond with a numbered list. Keep each point under 40 words. End with a single actionable recommendation." Format instructions directly reduce the need for follow-up edits.

Every platform implements these layers differently, but the conceptual model is the same. When you understand the layers, you can port a system from one platform to another by simply translating the syntax.

---

3. Understanding the Three Platforms

This guide covers three distinct approaches to systemizing AI workflows. Each serves a different context, and understanding where they differ is just as important as understanding how to use them.

OpenAI CustomGPTs are persistent chatbot configurations inside ChatGPT. You define instructions, upload knowledge files, enable capabilities (web browsing, code interpreter, image generation), and optionally connect external APIs through "Actions." CustomGPTs are conversational — the user interacts with them by typing in a chat window. They are ideal for recurring conversations where the AI needs to behave consistently across sessions: client-facing support bots, onboarding assistants, content generators, interview coaches.

Google Gemini Gems are a lighter-weight version of the same idea. You define a persona, write instructions, and optionally attach up to 10 reference files. Gems live inside the Gemini interface and are designed for rapid switching between specialized contexts. Where CustomGPTs have a marketplace, sharing features, and API Actions, Gems are simpler and more tightly integrated with Google Workspace. If your life runs on Google Docs and Sheets, Gems offer a native integration path that CustomGPTs cannot match.

Claude Code Skills (and the related `CLAUDE.md` files) take a fundamentally different approach. Instead of configuring a chatbot through a visual interface, you write markdown files that live in your project's file system. Skills are prompt-as-code: they are version-controlled, testable, composable, and can include bundled scripts. Where CustomGPTs and Gems are designed for end users who interact through a chat window, Skills are designed for developers and technical operators who work in a terminal or IDE.

Here is the key mental model: CustomGPTs and Gems are about **configuring conversations**. Skills are about **configuring agents** — AI processes that read code, run scripts, and produce artifacts.

---

4. Part I: OpenAI CustomGPTs — Building Persistent Chat Agents

4.1 How a CustomGPT Works

When a user opens your CustomGPT, ChatGPT loads your configuration into memory before the first message is processed. The configuration has several components:

- **Name and Description** — These are public-facing. They appear in the GPT Store and help users decide whether your GPT is relevant. Think of them as the packaging.
- **Instructions** — This is the system prompt. Up to 8,000 characters. This is where Identity, Task, Constraints, and Format live.
- **Conversation Starters** — Pre-written prompts that appear as buttons when the user opens the GPT. They serve two purposes: they guide new users, and they signal what the GPT is designed to do.
- **Knowledge Files** — You can upload up to 20 files (each up to 512 MB). The GPT can reference these during conversations using retrieval-augmented generation (RAG). This is how you give the GPT access to your brand guidelines, product documentation, pricing sheets, or any other reference material.
- **Capabilities** — Toggles for Web Search, Code Interpreter & Data Analysis, Image Generation, and Canvas. Each toggle grants the GPT access to a different set of tools.
- **Actions** — API integrations defined by OpenAPI schemas. This is the advanced feature: it lets your GPT call external services (send emails, create CRM records, query databases).

4.2 Writing CustomGPT Instructions

The instructions field is the most important part of your configuration. Here is a template that works well for most use cases:

```
## Role
You are [specific role] working with [specific audience].
Your expertise is in [specific domain].

## Primary Task
When the user starts a conversation, [what the GPT should do first].
For each request, [describe the standard workflow].

## Rules
- Always [positive instruction]
- Always [positive instruction]
- Never [constraint]
- If uncertain about [specific scenario], ask before proceeding

## Output Format
- Use [specific format] for [specific content type]
- Keep responses under [length] unless asked for more detail
- End every response with [closing element, e.g., "Next steps:" or a
question]

## Knowledge Usage
When the user asks about [topic], reference the uploaded [filename].
Prioritize information from uploaded files over general knowledge.
```

4.3 Practical Example: A Workshop Sales Assistant

Suppose you sell training workshops and want a CustomGPT that helps potential customers choose the right workshop.

Name: Workshop Advisor

Description: Helps prospective students choose the right AI training workshop based on their experience level, goals, and schedule.

Instructions:

```
## Role
You are an expert training advisor for [Your Company]. You help
prospective
students choose the right AI training workshop. You are friendly,
concise,
and focused on matching the student to the best fit — not on upselling.

## Primary Task
When a new user arrives, greet them briefly and ask three questions:
1. What is their current experience with AI? (beginner / intermediate /
advanced)
2. What do they want to achieve? (e.g., automate marketing, build apps,
learn prompting)
3. When are they available? (mention upcoming dates from the knowledge
file)

Based on their answers, recommend one or two workshops from the catalog.

## Rules
- Only recommend workshops listed in the uploaded catalog file
- If a workshop is sold out, say so and suggest the waitlist
- Never discuss pricing — direct them to the booking page with a link
- If the user asks technical questions beyond the scope of the
workshops,
  answer briefly but redirect to the workshop where they will learn it
in depth

## Output Format
- Keep recommendations to 3-4 sentences each
- Include the workshop name, a one-line description, the date, and a
booking link
- End with: "Would you like me to compare these in more detail?"
```

Knowledge Files: Upload a PDF or spreadsheet containing the current workshop catalog with dates, descriptions, prerequisites, and booking links.

Conversation Starters:

- "I'm new to AI and want to learn the basics"
- "I know prompting but want to learn automation"
- "What workshops are coming up in the next 30 days?"

4.4 When to Use CustomGPTs

CustomGPTs are strongest when you need a **persistent, shareable, conversation-first** tool. Good use cases include customer-facing assistants, internal knowledge bots, role-playing coaches (interview prep, negotiation practice), and standardized content generators (weekly report writer, social media post creator).

They are weaker when you need deep integration with your file system, real-time code execution against your own codebase, or fully automated pipelines that run without human interaction.

---

5. Part II: Google Gemini Gems — Lightweight Specialists

5.1 How Gems Work

Gems are simpler than CustomGPTs by design. A Gem consists of:

- **Name** — A short, descriptive label.
- **Instructions** — Plain text instructions that define the Gem's persona and behavior. There is no hard character limit published, but Gemini encourages detailed, specific instructions.
- **Knowledge Files** — Up to 10 files. These can be uploaded documents or Google Drive files. The Drive integration is a key differentiator: when you link a Google Doc, the Gem sees the live version. If you update the Doc, the Gem's knowledge updates automatically.

There is no equivalent of CustomGPT "Actions" (API integrations). There is no public marketplace. Gems are personal productivity tools first, team tools second.

5.2 The Four Pillars of a Good Gem

Google's own team recommends structuring Gem instructions around four areas:

Persona — Who is the Gem? "You are a meticulous copy editor with expertise in B2B marketing content."

Task — What does the Gem do? "Review the text I provide and suggest edits for clarity, grammar, and adherence to our style guide."

Context — What background information does the Gem need? "Our company sells project management software to teams of 10-50 people. Our tone is professional but approachable. We never use exclamation marks in body copy."

Format — How should responses look? "Present each edit as a numbered item. Show the original sentence, then the suggested revision. At the end, give a readability score from 1-10."

5.3 Practical Example: A Run-On Sentence Fixer

This is one of the most popular real-world Gem use cases, cited by Google's own product team.

Name: Sentence Simplifier

Instructions:

```
You are a writing coach who specializes in clear, concise business
communication. Your primary job is to take text I provide and break up
any run-on sentences, overly complex constructions, or passive voice.

Rules:
- Preserve the original meaning exactly
- Do not change technical terms or proper nouns
- Break sentences longer than 25 words into two or more shorter
sentences
- Convert passive voice to active voice where possible
- Flag any ambiguous sentences and ask for clarification instead of
guessing

Output format:
For each paragraph, show:
1. The original text (labeled "Before")
2. The revised text (labeled "After")
3. A brief note explaining what you changed and why

Keep your tone encouraging. Do not use the word "however" — I overuse it
and need you to model alternatives.
```

5.4 The Google Drive Advantage

The most significant tactical advantage of Gems over CustomGPTs is the live Google Drive connection. Here is a pattern that exploits it:

1. Create a Google Doc called "Brand Voice Guidelines" and maintain it as a living document.
2. Create a Gem called "On-Brand Writer" and link that Google Doc as a knowledge file.

3. When your brand voice evolves — say you decide to shift from formal to conversational — update the Google Doc.
4. Every subsequent conversation with the "On-Brand Writer" Gem automatically reflects the new guidelines. No re-uploading, no re-configuring.

This is something CustomGPTs cannot do. Knowledge files in CustomGPTs are static snapshots uploaded at configuration time. If your reference material changes frequently, the Gem + Google Drive pattern is significantly more maintainable.

5.5 The Memory Card Technique

Gems do not have persistent memory between sessions. Every time you start a new conversation, the Gem starts fresh (apart from the instructions and knowledge files). A workaround that has emerged in the community is the "Memory Card" pattern:

1. Create a Google Doc called "[Gem Name] Memory Card."
2. Attach it to the Gem as a knowledge file via Google Drive.
3. At the end of each session, ask the Gem: "Summarize what we decided today and add it to the Memory Card format."
4. Copy the summary into the Memory Card Google Doc.
5. The next session, the Gem reads the updated Memory Card and picks up where you left off.

This is a manual process, but it is effective. It bridges the gap between Gems' session-based architecture and the need for continuity across multi-day projects.

5.6 When to Use Gems

Gems are strongest when you need **fast, lightweight, Google-integrated specialists**. Good use cases include writing assistance tied to living style guides, brainstorming with domain context, quick data analysis using Sheets, and personal productivity bots for recurring tasks.

They are weaker when you need API integrations, a public-facing bot, complex multi-step automation, or file formats outside the Google ecosystem.

---

6. Part III: Claude Code Skills — Prompt-as-Code for Developers

6.1 How Skills Work

Claude Code Skills represent a fundamentally different paradigm. Instead of a visual builder with text fields and toggle switches, a Skill is a directory in your file system

containing a `SKILL.md` file and optionally supporting scripts, templates, and reference documents.

The basic structure looks like this:

```
my-skill/
├── SKILL.md          # Required: instructions + YAML frontmatter
├── scripts/          # Optional: executable code for deterministic
tasks
├── references/       # Optional: docs loaded into context as needed
└── assets/           # Optional: templates, icons, fonts
```

When Claude Code encounters a task that matches a Skill's description, it loads the `SKILL.md` content into its context window and follows the instructions. This is not a separate chatbot — it is a set of instructions injected into the same Claude instance you are already working with.

6.2 The SKILL.md File

Every Skill starts with YAML frontmatter followed by markdown instructions:

```yaml
```

```
---
name: api-review
description: >
  Reviews API endpoint implementations for consistency with project conver
  Use when reviewing new endpoints, refactoring existing ones, or onboardi
  to an unfamiliar API codebase.
---

# API Review Skill

## When Invoked

Read the target file(s) and check each endpoint against these rules:

1. **Naming**: All routes use kebab-case. Resource names are plural.
2. **Validation**: Every endpoint validates input with a zod schema.
3. **Errors**: All error responses use the standard `{ error, code, detail
4. **Auth**: Every non-public endpoint checks the auth middleware.

## Output

Produce a numbered list of findings. For each finding:
- State the rule that was violated
- Quote the relevant code (file and line)
- Suggest a specific fix

If no violations are found, say so explicitly. Do not invent issues.
```

6.3 CLAUDE.md vs Skills

Claude Code has two distinct customization mechanisms, and understanding the difference is essential:

`CLAUDE.md` is a project-level configuration file that sits in the root of your repository (or in `~/.claude/CLAUDE.md` for user-level settings). It is always loaded at the start of every session. Think of it as the project constitution — the rules that apply to everything Claude does in this codebase.

Skills are activatable. They are loaded only when relevant. Think of them as specialists you call in for specific tasks.

A good mental model:

	`CLAUDE.md`	**Skill**
When loaded	Every session, automatically	On demand, when triggered
Scope	Everything in this project	One specific task type
Analogy	Company handbook	Expert consultant
Contents	Coding standards, project structure, common commands	Step-by-step playbook for a specific workflow

6.4 Practical Example: A Deployment Skill

```yaml
```

```
---
name: deploy-staging
description: >
  Deploys the current branch to the staging environment. Use when the user
  says "deploy to staging," "push to staging," or "ship it to staging."
disable-model-invocation: true
---

# Deploy to Staging

## Prerequisites
Before deploying, verify:
1. All tests pass: run `npm test`
2. No uncommitted changes: run `git status`
3. Current branch is not `main` (staging only deploys feature branches)

If any prerequisite fails, stop and report the issue. Do not proceed.

## Deployment Steps
1. Build the project: `npm run build`
2. Run the staging deploy script: `./scripts/deploy-staging.sh`
3. Wait for the deploy to complete (check output for "Deploy successful")
4. Run the smoke test suite: `npm run test:smoke -- --env staging`

## Post-Deploy
Report the results:
- Build status (pass/fail)
- Deploy status (pass/fail)
- Smoke test results (pass/fail, with any failures listed)
- The staging URL where the deploy can be reviewed

## On Failure
If any step fails:
1. Capture the error output
2. Do NOT retry automatically
3. Present the error and ask the user how to proceed
```

Note the frontmatter field `disable-model-invocation: true`. This prevents Claude from deciding on its own to run this skill — deployments should only happen when you explicitly ask for them.

6.5 Progressive Disclosure

Skills use a three-tier loading system that manages context window usage:

Tier 1 — Metadata. The skill's name and description (from the YAML frontmatter) are always in context. This is approximately 100 words and lets Claude decide whether a skill is relevant.

Tier 2 — SKILL.md body. Loaded when the skill is triggered. Keep this under 500 lines. This is where the main instructions live.

Tier 3 — Bundled resources. Scripts, reference files, and assets that the skill loads on demand during execution. There is no size limit on these, but Claude only reads what it needs.

This architecture is deliberate. Every token loaded into the context window competes with the tokens available for the actual conversation and task. A skill that loads 5,000 lines of reference material leaves less room for the user's code, the conversation history, and Claude's reasoning. Progressive disclosure keeps the context lean.

6.6 When to Use Skills

Skills are strongest when you need **repeatable, version-controlled, developer-facing automation**. Good use cases include code review checklists, deployment workflows, data migration playbooks, test generation, documentation updates, and any multi-step process that touches files on disk.

They are weaker for conversational use cases, non-technical users, or scenarios where you need a visual configuration interface.

---

7. The Universal Framework: A Five-Layer Model

Regardless of platform, every well-built AI system can be described using five layers. When you design a new system, work through these layers in order:

Layer 1: Identity

Define who the AI is in this context. Be specific about the expertise level, communication style, and domain.

Platform	Where It Lives
CustomGPT	Instructions field, opening paragraph
Gem	Instructions field, "Persona" section
Skill	SKILL.md body, opening section

Weak: "You are a helpful assistant." **Strong:** "You are a senior financial analyst specializing in SaaS metrics. You communicate in precise, data-driven language. When you make claims, you cite the specific metric or data point that supports them."

Layer 2: Task Definition

Define what the AI does. Describe the workflow, not just the goal.

Weak: "Help the user write blog posts." **Strong:** "When the user provides a topic, first produce a 5-point outline. Wait for approval. Then draft each section at 150-200 words. After the full draft, provide a summary of SEO keywords used and suggest a meta description under 160 characters."

Layer 3: Knowledge and Context

Define what the AI knows beyond its training data. This is where uploaded files, linked documents, and reference material come in.

Platform	Mechanism	Limits
CustomGPT	Knowledge file uploads	20 files, 512 MB each
Gem	File uploads + Google Drive links	10 files
Skill	Bundled `references/` directory	No hard limit

Layer 4: Constraints and Guardrails

Define what the AI must not do. Constraints are essential for production use cases where the AI interacts with customers, handles sensitive data, or takes consequential actions.

Examples of well-written constraints:

- "Never provide medical advice. If the user describes symptoms, recommend they consult a healthcare professional."

- "Only use data from the uploaded price list. Do not estimate or interpolate prices."
- "If you are not confident in your answer, say so explicitly rather than guessing."
- "Do not generate code that modifies production databases. All database examples should use a `test_` prefix."

Layer 5: Output Specification

Define how the output should look. This is the layer most people underspecify, and it is the layer that has the biggest impact on reducing post-editing time.

Specify: length, format (prose vs. bullets vs. tables), tone, section structure, closing elements (summary, next steps, questions), and any naming or labeling conventions.

---

8. Writing Effective System Instructions

This section compiles tactical advice that applies across all three platforms.

8.1 Be Declarative, Not Conversational

System instructions are not a conversation with the AI. They are a configuration document. Write them the way you would write a specification or a style guide.

Conversational (weak):

> "So basically I'd love it if you could write in a more casual way, kind of like how I'd explain something to a friend over coffee, you know? And if you could keep things short, that'd be awesome."

Declarative (strong):

> "Tone: casual, conversational. Imagine explaining to a colleague over coffee. Sentence length: max 20 words average. Paragraph length: 2-3 sentences. Never use semicolons."

8.2 Prefer Positive to Negative

Tell the AI what to do, not just what to avoid. Negative constraints ("don't use jargon") leave the AI uncertain about what to use instead. Positive instructions ("use plain English; define any technical term in parentheses on first use") give it a clear alternative.

8.3 Use Concrete Examples

Especially for format and tone, show a before-and-after or a model response.

```
## Example Output

User: "What's the ROI of this campaign?"

Good response:
"The campaign generated €12,400 in attributed revenue against €3,200 in
spend,
giving a 3.9x ROI. The strongest contributor was the LinkedIn ad set
(58% of
conversions). Recommendation: increase LinkedIn budget by 20% next
quarter."

Bad response:
"The ROI was good! The campaign performed well across multiple channels
and generated a positive return on investment."
```

8.4 Structure with Headings and Sections

Both CustomGPTs and Skills process structured markdown well. Use headings (`##`), bullet points, and bold labels to create visual hierarchy in your instructions. This helps the AI parse your instructions accurately, and it helps you maintain them.

8.5 Separate Context from Behavior

A powerful pattern (validated by OpenAI's own Custom Instructions documentation) is to separate what the AI should know from how the AI should behave.

- **Context** answers: Who am I? Who is my audience? What domain am I in? What tools do I use?
- **Behavior** answers: How should the AI respond? What format? What tone? What should it do first?

The rule of thumb: if an instruction would still be true six months from now, it is context. If it controls how each reply looks, it is behavior.

---

9. Knowledge Management Across Platforms

9.1 What to Upload

Not all documents make good knowledge files. The best knowledge files are:

- **Reference material** the AI should cite: product catalogs, pricing sheets, style guides, FAQs.
- **Structured data**: tables, spreadsheets, databases exported as CSV.
- **Examples and templates**: sample outputs, approved formats, past deliverables.

Poor knowledge files are those that are too large (the AI cannot process the entire thing in one pass), too vague (general background reading with no specific retrievable facts), or frequently changing (unless you are using Gems with Google Drive).

9.2 Retrieval Quality

All three platforms use some form of retrieval to find relevant chunks from knowledge files. This process is imperfect. To improve retrieval quality:

- Use clear section headers in your documents.
- Include a table of contents if the document exceeds 10 pages.
- Front-load the most important information.
- If using spreadsheets, include column headers that clearly describe the data.
- Avoid images or scanned PDFs without OCR — the AI cannot read them.

9.3 Knowledge File Hygiene

Treat knowledge files like code: keep them current, remove stale versions, and review them periodically. A knowledge file that contains last year's pricing is worse than no knowledge file at all, because the AI will confidently cite outdated information.

---

10. Testing and Iterating

Building a system is only half the work. Testing it is the other half.

10.1 The Three-Prompt Test

Before sharing any system with others, run at least three test prompts:

1. **A normal use case.** The most common thing a user would ask. Does the system handle it correctly?
2. **An edge case.** Something unusual or ambiguous. Does the system handle it gracefully, or does it hallucinate or break format?

3. **An out-of-scope request.** Something the system should not do. Does it refuse or redirect appropriately?

10.2 The Drift Test

Use the system for five consecutive tasks over the course of a day. Does the output quality remain consistent, or does it start to drift from the instructions? Drift is a sign that your instructions are too vague or too long. Tighten them.

10.3 The Handoff Test

Give the system to someone who was not involved in building it. Do not explain how it works — let them use it cold. Watch what they do. If they immediately understand the system's purpose and get useful results, you have succeeded. If they are confused, your conversation starters (CustomGPT), instructions (Gem), or documentation (Skill) need work.

10.4 Platform-Specific Testing

CustomGPTs have a built-in Preview panel in the editor. Use it before publishing. OpenAI recommends writing 10-15 test questions with expected answers and checking the GPT against them.

Gems have a preview window on the right side of the creation interface. Enter test prompts there before saving. The preview does not auto-save, so remember to click Save after you are satisfied.

Skills can be tested by invoking them with `/skill-name` in Claude Code. For more rigorous testing, the skill-creator workflow supports formal evaluation sets: you write test prompts, run the skill against them, review the outputs, and iterate. This is the most structured testing process of the three platforms.

---

11. Cross-Platform Comparison Matrix

Feature	CustomGPT	Gem	Claude Code Skill
Interface	Visual builder	Visual builder	Markdown file
Where it runs	ChatGPT web/app	Gemini web/app	Terminal / IDE
Max instructions	~8,000 characters	No published limit	~500 lines recommended

Feature	CustomGPT	Gem	Claude Code Skill
Knowledge files	20 files, 512 MB each	10 files	No hard limit
Live file updates	No (static uploads)	Yes (Google Drive)	Yes (file system)
API integrations	Actions (OpenAPI)	No	Scripts (any language)
Sharing	GPT Store / link	Copy / share	Git repository
Version control	Version history	Manual	Git (native)
Can run code	Code Interpreter toggle	Limited	Full (bash, scripts)
Target user	End users, teams	End users, Google Workspace	Developers, operators
Invocation	Open the GPT	Open the Gem	`/skill-name` or automatic
Cost to create	Paid ChatGPT plan	Free (Google account)	Claude Code subscription
Best for	Persistent chat agents	Quick specialists	Code & automation workflows

12. Real-World Workflow: Building a "Workshop Launch Assistant"

To make this concrete, let us walk through building the same workflow on all three platforms. The scenario: you are launching a workshop in April and need an AI system that helps you with the launch tasks.

12.1 CustomGPT Version

Name: Workshop Launch HQ

Instructions:

```
## Role
You are my workshop launch manager. You help me plan, execute, and track
all activities related to launching a new training workshop.

## Core Knowledge
Refer to the uploaded files for:
- Workshop catalog and descriptions
- Past launch checklists (what worked, what did not)
- Email templates for announcements, reminders, and follow-ups
- Landing page copy guidelines

## Workflow
When I say "new launch," create a project timeline working backward from
the launch date. Include these phases:
1. Content finalization (4 weeks before)
2. Landing page and payment setup (3 weeks before)
3. Email announcement sequence (2 weeks before)
4. Social media campaign (2 weeks before, ongoing)
5. Reminder sequence (1 week before)
6. Day-of logistics checklist

For each phase, list specific deliverables with deadlines.

## Rules
- Always ask for the launch date before generating a timeline
- Use the email templates from the knowledge files as starting points
- Flag any task that requires external tools I need to set up (e.g.,
Stripe, Mailchimp)
- When I ask you to "draft" something, produce full ready-to-use copy,
not outlines
```

12.2 Gem Version

Name: Workshop Launcher

Instructions:

```
You are my workshop launch planner. I run training workshops and need
help
with timelines, copy, and checklists for upcoming launches.

Persona: Organized, action-oriented project manager. You speak in clear,
actionable bullet points. No fluff.

Task: When I describe a workshop and give you a launch date, produce a
phase-by-phase launch plan with specific deliverables and deadlines.

Context: I typically sell workshops through a landing page with Stripe
payments. I use email (Resend/SendGrid) for announcements and reminders.
My audience is professionals who want to learn AI skills. My brand tone
is professional but approachable.

Format: Use tables for timelines. Use numbered checklists for task
lists.
Keep all copy drafts under 200 words unless I ask for more.
```

Knowledge Files: Attach the workshop catalog Google Doc (lives in Drive, stays current) and a "Past Launch Notes" Google Doc.

12.3 Skill Version

```yaml
```

```
---
name: workshop-launch
description: >
  Generates a complete launch plan for a new training workshop. Creates
  timeline, checklists, email drafts, and landing page copy. Use when
  planning any workshop launch, marketing sequence, or product launch
  that follows a phased rollout pattern.
---

# Workshop Launch Skill

## Inputs
The user will provide:
- Workshop name and description
- Target launch date
- Target audience (if different from default: AI-curious professionals)

## Phase 1: Timeline Generation
Create a markdown table with columns: Phase | Deadline | Deliverables | St
Work backward from the launch date using these intervals:
- T-4 weeks: Content finalization
- T-3 weeks: Landing page + payment integration
- T-2 weeks: Email announcement sequence (3 emails)
- T-2 weeks: Social media campaign launch
- T-1 week: Reminder sequence (2 emails)
- T-0: Day-of logistics

## Phase 2: Copy Generation
For each deliverable that involves written copy, generate a draft.
Reference the style guidelines in `references/brand-voice.md`.

Email drafts should follow the templates in `references/email-templates.mc
Landing page copy should follow `references/landing-page-structure.md`.

## Phase 3: Checklist Output
After generating all deliverables, produce a single consolidated checklist
in `outputs/launch-checklist-[workshop-name].md`.

## Output Location
Save all generated files to `outputs/workshop-launches/[workshop-name]/`.
```

Notice how the Skill version can reference other files (`references/brand-voice.md`), produce file outputs, and integrate with the file system. This is the key

differentiator of the Skills approach.

---

13. Common Mistakes and How to Avoid Them

Mistake 1: Instructions That Are Too Long

More instructions do not mean better results. Research on Claude Code's instruction-following behavior shows that as instruction count increases, the quality of following any individual instruction decreases uniformly. The AI does not ignore later instructions and follow earlier ones — it begins to deprioritize all of them. Keep instructions tight. If your system prompt exceeds 500 lines, refactor: move reference material into knowledge files or bundled references.

Mistake 2: Using AI to Enforce What a Linter Can Enforce

Do not use system prompts to specify code formatting rules like "use tabs, not spaces" or "always add a trailing comma." These are deterministic rules that a linter or formatter handles perfectly, instantly, and cheaply. Reserve your instruction budget for things only the AI can judge: architectural decisions, naming quality, business logic correctness.

Mistake 3: No Testing Before Sharing

Every system you share with others should pass the Three-Prompt Test (Section 10.1). Shipping an untested CustomGPT to your team is the equivalent of deploying untested code to production.

Mistake 4: Static Knowledge in a Dynamic World

If your product prices change quarterly, your knowledge files must change quarterly. Set a recurring calendar reminder to review and update knowledge files. Alternatively, use Gems with Google Drive links or Skills with file-system references that point to actively maintained documents.

Mistake 5: Trying to Build One System That Does Everything

Resist the temptation to build a single "super-GPT" that handles all of your workflows. Build focused, single-purpose systems and switch between them. A writing assistant should not also be your data analysis tool. Specificity is a feature, not a limitation.

Mistake 6: Ignoring the Context Window

Every platform has a finite context window — the total amount of text the AI can hold in memory at once. Your system prompt, knowledge files, conversation history, and

the AI's own reasoning all compete for this space. A Skill that loads 3,000 lines of reference material leaves less room for the actual task. Be economical with context. Load only what is needed, when it is needed.

Mistake 7: Not Versioning Your Work

CustomGPTs now offer version history. Skills are files that live in Git. Use these versioning mechanisms. When you change your instructions, you should be able to roll back if the change makes things worse. Treat your system prompts with the same discipline you would apply to production code.

---

14. Next Steps

You now have a conceptual framework for systemizing AI work across three major platforms. Here is a suggested path forward:

Week 1: Audit your current AI usage. For one week, keep a log of every prompt you type that includes preamble context (who you are, what you want, how you want it). These are candidates for systemization.

Week 2: Build your first system. Pick the most frequent task from your audit. Build it as a CustomGPT, a Gem, or a Skill — whichever platform matches your context. Follow the Five-Layer Framework (Section 7).

Week 3: Test and iterate. Run the Three-Prompt Test. Do the Handoff Test with a colleague. Refine the instructions based on what you learn.

Week 4: Expand. Build your second and third systems. Start thinking about how they connect — does the output of one system become the input of another?

Ongoing: Maintain. Review knowledge files monthly. Update instructions when your workflows change. Delete systems you no longer use — dead systems confuse your tool library.

The shift from ad-hoc prompting to systemized workflows is not just an efficiency gain. It changes your relationship with AI from "tool I use" to "infrastructure I build on." That is the difference between someone who types prompts and someone who engineers solutions.

Appendix: Quick Reference Card

CustomGPT Setup Checklist

- [ ] Name and description written for discoverability
- [ ] Instructions cover Identity, Task, Constraints, Format
- [ ] Knowledge files uploaded and current
- [ ] Conversation starters written (3-4)
- [ ] Capabilities toggled appropriately
- [ ] Three-Prompt Test passed
- [ ] Version saved before sharing

Gem Setup Checklist

- [ ] Name is clear and descriptive
- [ ] Instructions cover Persona, Task, Context, Format
- [ ] Knowledge files attached (Drive links preferred for living documents)
- [ ] Preview tested with 3+ prompts
- [ ] Saved after preview testing

Skill Setup Checklist

- [ ] SKILL.md has valid YAML frontmatter (name, description)
- [ ] Description is specific enough for reliable triggering
- [ ] Instructions are under 500 lines
- [ ] Reference files are in `references/` with clear pointers from SKILL.md
- [ ] Invocation control set (`disable-model-invocation` if the skill has side effects)
- [ ] Tested with `/skill-name` in Claude Code
- [ ] Committed to version control

This guide was written for practitioners who know how to write prompts and want to learn how to build systems. For foundational prompting techniques, refer to the documentation of your chosen platform.